Foreign Whispers:
Final Project Report

Ahmed Jaheen

May 4, 2026

Abstract

This report presents the implementation of *Foreign Whispers*, an open-source video dubbing pipeline that transforms an English YouTube video into a dubbed Spanish output using local components. The final system supports video download, transcription, translation, speaker diarization, speaker-aware voice selection, time-aware text-to-speech, and final audio/video stitching. The project was completed with an emphasis on practical usability: the codebase now runs through a browser-based studio, exposes a FastAPI backend, supports a Docker GPU workflow, and also includes a local macOS CPU fallback path for development and validation. This report summarizes the architecture, the main technical decisions, the implemented notebook tasks, debugging work, and the final deliverables.

1 Project Goal

The goal of Foreign Whispers is to recreate the essential stages of a modern video dubbing system using open-source tools and local execution instead of paid APIs. The pipeline accepts a YouTube URL, downloads the source video and captions, transcribes spoken audio, translates the transcript into Spanish, synthesizes a dubbed audio track, and stitches that audio back into the source video while generating subtitle tracks for playback in the frontend.

Compared with a simple translation demo, this project is harder because each stage has timing consequences for the next stage. A translation that is semantically correct can still fail if it is too long to be spoken in the original window. Likewise, text-to-speech audio that sounds natural can still produce a poor video if it drifts away from the visible speaker. The final implementation therefore focuses not only on correctness at each stage, but also on timing stability across the entire pipeline.

2 System Architecture

The final system uses a layered architecture with a browser frontend, a FastAPI orchestrator, optional GPU inference services, and a reusable Python library.

2.1 Major Components

- **Frontend (Next.js):** The Dubbing Studio runs on port 8501 and provides video selection, pipeline settings, stage monitoring, result playback, and caption rendering.
- **Backend (FastAPI):** The API runs on port 8080 and exposes the pipeline routes for download, transcription, diarization, translation, TTS, stitching, and media serving.

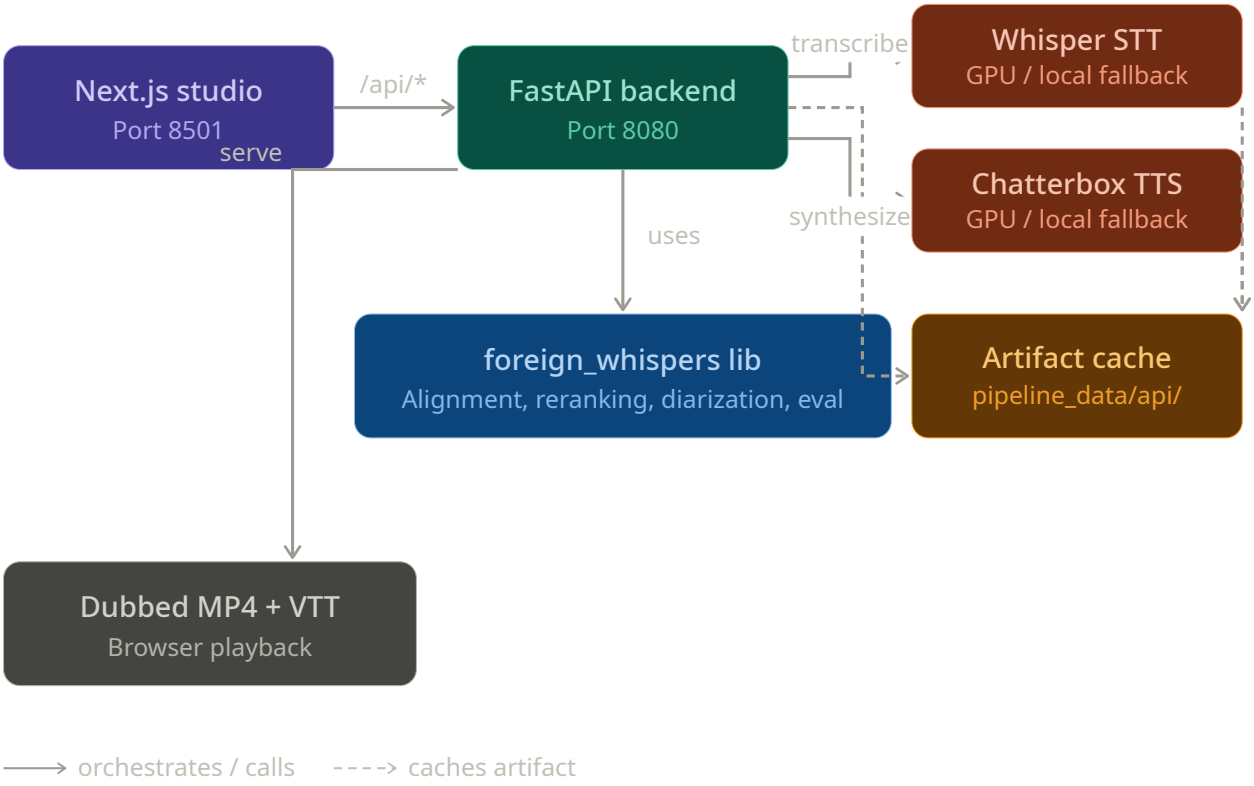

Figure 1: High-level architecture of the Foreign Whispers pipeline. The browser frontend calls the FastAPI backend, which orchestrates the pipeline stages, delegates model-heavy work to inference backends, and caches all intermediate and final artifacts under `pipeline_data/api/`.

- **Library (`foreign_whispers`):** This package contains the alignment logic, evaluation helpers, reranking code, diarization helpers, and voice resolution functions. It is intentionally reusable outside the web app.
- **Inference Services:** In the intended GPU deployment, Whisper and Chatterbox run in dedicated services. In local macOS development, the backend can fall back to CPU-safe implementations.
- **Artifact Cache:** All pipeline outputs are stored under `pipeline_data/api/`. This makes repeated runs faster and supports inspection of each intermediate stage.

2.2 Pipeline Flow

The full processing sequence is:

1. Download the YouTube video and captions.
2. Transcribe the video audio using Whisper.
3. Diarize the audio to determine which speaker is talking in each region.
4. Translate the transcript from English to Spanish.
5. Resolve voices and synthesize Spanish audio with timing control.
6. Stitch the dubbed audio and WebVTT subtitles back into the final video.

The frontend calls the backend step by step, and the backend stores the artifacts of every stage so that debugging and reruns are manageable.

3 Implementation Details

3.1 Download and Transcription

The download stage uses `yt-dlp` to retrieve the source MP4 and YouTube captions. The transcription stage uses Whisper. One important practical issue was that YouTube captions are often too coarse for dubbing because they cover long windows rather than short spoken phrases. For that reason, the frontend defaults to Whisper timing instead of YouTube caption timing.

Additional segmentation logic was used to split overlong transcription windows into smaller phrase-like segments. This significantly improved alignment and reduced the silence-heavy behavior observed in early runs.

3.2 Translation and Duration-Aware Reranking

The translation stage uses Argos Translate for offline English-to-Spanish translation. However, direct translation alone is not enough, because Spanish often expands relative to English. To handle this, the project implements duration-aware translation reranking in `foreign_whispers/reranking.py`.

The reranking helper generates shorter alternatives when a translated segment is too long for its target timing window. This improvement is reused in both the translation notebook and the alignment notebook. In practice, it reduced the number of severe stretch cases and helped prevent missing words in TTS.

3.3 Alignment and Timing Control

The alignment work was one of the most important parts of the project. The initial heuristic estimated TTS duration mostly from character count, which was too crude. The improved implementation uses a better duration estimate and a stronger global alignment strategy to schedule translated segments more intelligently.

Another important refinement was distinguishing *natural pacing* from *forced stretching*. In baseline mode, the system now preserves natural speech and allows drift. In aligned mode, it tries to fit speech to the video window while avoiding overly slow or heavily clipped audio. Short segments are no longer artificially slowed just to fill the whole subtitle window; instead, the system prefers silence padding when that sounds better.

3.4 Diarization and Speaker Assignment

The diarization stage was implemented with `pyannote.audio`. This stage extracts audio from the source video, runs diarization, caches the speaker segments, and merges speaker labels back into the transcript. The core merging logic assigns each transcription segment to the speaker with the strongest temporal overlap.

Because the project also needed male and female speaker-aware dubbing, the implementation builds lightweight speaker profiles from pitch estimates over the diarized audio regions. Those profiles are not full biometric models; they are simple cues used to resolve an appropriate male or female reference voice. This is sufficient for the course requirement that a woman should sound like a woman and a man should sound like a man.

3.5 Voice Resolution and TTS

Voice resolution was implemented in `foreign_whispers/voice_resolution.py`. The resolution order is:

1. speaker-specific WAV
2. language-specific gender default
3. language default
4. global default

This design makes the system robust. When a dedicated speaker reference file is available, it is used. Otherwise, the pipeline can still choose a male or female language default. For Spanish, reference WAV files were added under `pipeline_data/speakers/es/`.

In the intended Docker GPU setup, TTS is served by Chatterbox. In local macOS development, however, Chatterbox may not be available. To keep the project fully runnable, CPU-safe fallbacks were added. A major late-stage bug was that the local fallback used a single generic voice, which made correct diarization sound wrong to the user. This was fixed by routing male and female diarized segments to distinct local macOS Spanish voices when Chatterbox is unavailable.

3.6 Stitching and Captions

The stitch stage remuxes the synthesized audio back into the source MP4 using `ffmpeg` without re-encoding the video track. This keeps the output fast and high quality. The system also serves WebVTT subtitle files for both original and translated captions.

A caption rendering bug was discovered where translated text appeared twice on top of itself in the player. The root cause was overlapping cue behavior from a carry-over caption format. The stitch code was updated so the frontend now receives clean one-cue subtitle timing for the translated captions.

4 Debugging and Engineering Improvements

Several practical issues had to be solved before the application became stable.

4.1 Cache Invalidation

The pipeline caches intermediate results aggressively. This is useful for performance, but it created a class of bugs where later code improvements did not appear in the output because old translation or TTS files were reused. The backend was updated to invalidate caches when speaker labels or speaker voice metadata were missing from downstream artifacts.

4.2 Frontend Pipeline State

Another issue appeared in the frontend stage table: runs could look as if multiple stages were executing out of order. The root cause was overlapping or stale asynchronous updates from different runs. The pipeline hook was fixed so only the current run can update stage state, and overlapping runs are now blocked.

4.3 Settings Dependency Handling

The user also observed that the diarization stage was being skipped even after selecting speaker-aware TTS. This came from a frontend settings dependency problem. The fix was to make diarization a derived requirement when voice cloning is enabled, so the UI and pipeline logic stay consistent.

5 Evaluation and Testing

The project was validated with both automated tests and real pipeline runs. The backend test suite passes successfully, and frontend lint/build checks were also used after UI changes. In addition, several end-to-end reruns were used to verify that early timeline regions no longer contained long silence windows, that subtitle duplication was fixed, and that speaker-aware male/female voice assignment worked in practice.

The final codebase therefore supports both analytical testing and practical user-facing verification through the browser interface.

6 Deliverables

6.1 Codebase and README

The repository now contains:

- the full application code
- exact Docker and macOS local run instructions

- environment variable guidance
- output artifact locations
- validation commands for tests, lint, and build checks

6.2 Report

This LaTeX file is intended as the report deliverable

6.3 Demo Video and Sample Input/Output

The demo video and sample input/output videos are separate deliverables in a google drive attached in the submission.

7 Conclusion

The final Foreign Whispers implementation goes beyond a basic proof of concept. It now behaves like a usable end-to-end dubbing system with a frontend studio, an API backend, reusable alignment logic, diarization support, speaker-aware voice selection, caching, and stable local development workflows.

The most important lesson from the project is that dubbing quality depends on the interaction between components, not just the quality of any one model. Translation length, segment timing, speaker assignment, and TTS behavior all affect the final video. The completed system addresses these interactions through alignment-aware translation, cache correctness, speaker-aware fallback voices, and a more reliable frontend execution model.

I went through ALOT OF ERRORS, which helped me understanding the real world enviroment as AI Engineer in a big project, and how to deal with it.